SOFTWARE DESIGN & ARCHITECTURE

Group Activity — Conceptual Classes and Abstraction

Selected Domain	Library Management System
Main scope	Books, members, borrowing, reservations, overdue fines and memberships.
Activity tasks	Candidate classes • Attributes and methods • Abstraction decisions • Invented software object

Purpose

This activity models a library system using object-oriented design. It identifies the important concepts, separates data from behavior, and explains why software models do not need to copy every detail of the real world.

Chosen domain: A. Library System

The system helps members find books, borrow and return copies, reserve unavailable books, and calculate overdue fines. Librarians can manage book records and memberships.

1. Candidate Conceptual Classes

Both noun-phrase and category-list techniques are used to identify the main classes.

A. Noun-phrase technique

Key nouns from the problem description include **book**, **member**, **librarian**, **loan**, **reservation**, **fine**, **membership**, and **catalogue**. These nouns are candidates for classes because the system stores information about them or performs actions involving them.

B. Category-list technique

Category	Candidate class	Reason
People / roles	Member, Librarian	People use or manage the library system.
Things / resources	Book, BookCopy, Catalogue	Books are searched, stored and borrowed.
Transactions	Loan, Reservation	The system records borrowing and reservation activity.
Rules / charges	Fine, Membership	The system tracks membership status and overdue charges.

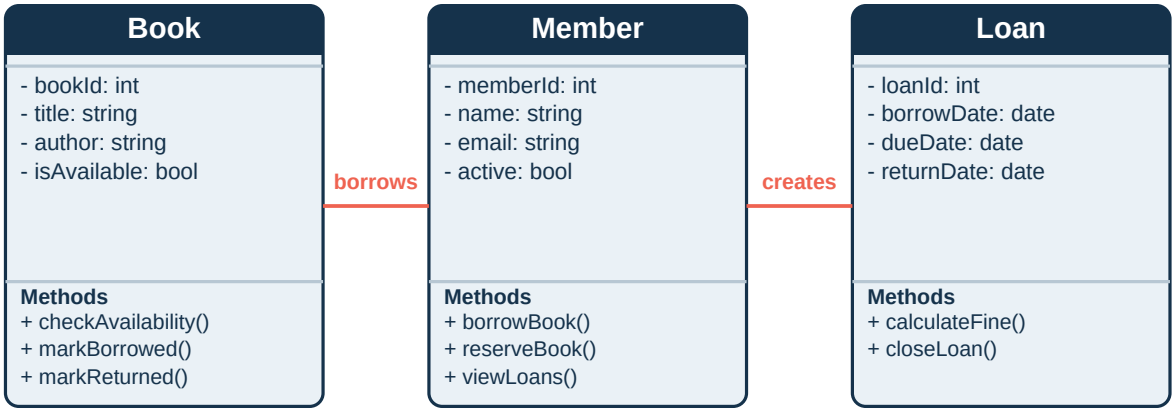
Candidate class list (10)

1. **Book** — Title, author, ISBN and publication information.
2. **BookCopy** — A physical or lendable copy of a book.
3. **Member** — A registered person who borrows or reserves books.
4. **Librarian** — A staff user who manages books and member records.
5. **Loan** — A borrowing record with borrow and due dates.
6. **Reservation** — A request to borrow a book when it becomes available.
7. **Fine** — A charge associated with an overdue loan.
8. **Membership** — The member's registration type, validity and status.
9. **Catalogue** — The searchable collection of book records.
10. **Notification** — A message about due dates, availability or overdue items.

Selected core classes for detailed design: Book, Member and Loan.

2. Attributes and Methods for Three Important Classes

Attributes describe an object's state (what it knows). Methods describe behavior (what it can do). The three core classes below cover the book, the person borrowing it, and the borrowing transaction.



Class	Attributes (state)	Methods (behavior)
Book	bookId, title, author, ISBN, isAvailable	checkAvailability(), markBorrowed(), markReturned()
Member	memberId, name, email, active	borrowBook(), reserveBook(), viewLoans()
Loan	loanId, borrowDate, dueDate, returnDate	calculateFine(), closeLoan()

How the classes work together

A Member requests a BookCopy. The system creates a Loan containing the borrow date and due date. When the copy is returned, the Loan is closed and any overdue Fine can be calculated. In a full implementation, Book represents a title while BookCopy represents each lendable copy.

3. Abstraction Decisions

Abstraction means keeping the details that matter for the system's purpose and leaving out details that do not help the required tasks.

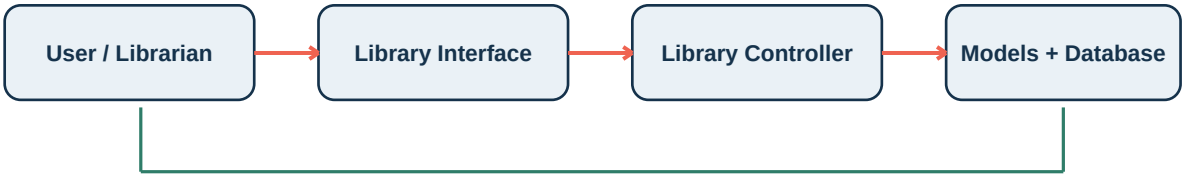
For the library system, the Book class stores useful details such as title, author, ISBN and availability. We deliberately ignore details such as the book's cover colour, paper texture, exact weight and the shelf's appearance because these are not needed to search, borrow or return a book.

The Member class stores a member ID, name, contact email and membership status. We do not model personal hobbies, clothing, height or other unrelated facts, because they do not affect borrowing rules.

The Loan class records borrowing and due dates, return date and fine-related information. It does not simulate the member's journey through the building or every conversation with library staff. These decisions keep the model simple, understandable and focused on the required functions.

Abstraction diagram: real world → relevant details → software model

Real-world object	Keep (relevant)	Ignore (not needed)	Software class
A book	Title, author, ISBN, availability	Cover colour, paper texture	Book
A library member	Member ID, name, status	Hobbies, clothing, height	Member
A borrowing event	Borrow date, due date, return date	Journey, conversations	Loan



4. One “Invented” Software Object

Invented object: FineCalculator

A FineCalculator is a software object that calculates an overdue fine from a loan's due date, return date and the library's daily fine rate. It is not a physical object found in a real library; it is a useful software abstraction that keeps calculation rules in one place.

Possible attributes

• dailyFineRate: decimal
• maximumFine: decimal (optional)
• gracePeriodDays: integer

Possible methods

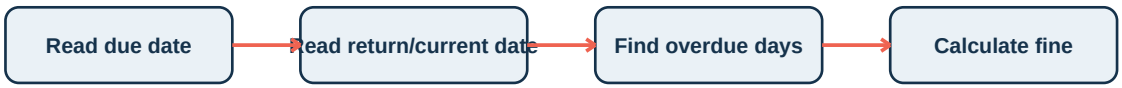
• calculateFine(dueDate, returnDate)
• isOverdue(dueDate, currentDate)
• applyFineLimit(amount)

Example calculation

If the due date has passed by 4 days and the library charges Rs. 20 per overdue day, the fine is 4 × Rs. 20 = Rs. 80, assuming no grace period or maximum-fine rule.

Input	Rule	Output
4 overdue days	4 × Rs. 20 per day	Rs. 80 fine

FineCalculator flow diagram



Fine = overdue days × daily rate

Then apply any grace period or maximum-fine rule

Conclusion

The Library Management System can be represented with a small set of meaningful classes. Attributes store state, methods represent behavior, and abstraction removes irrelevant real-world details. A software-only object such as FineCalculator helps centralize a rule and makes the system easier to maintain and test.

Note: The fine rate in the example is illustrative; a real library would use its own policy.